

Вложенные классы

Абстрактные классы

Интерфейсы

Перечисления



План урока

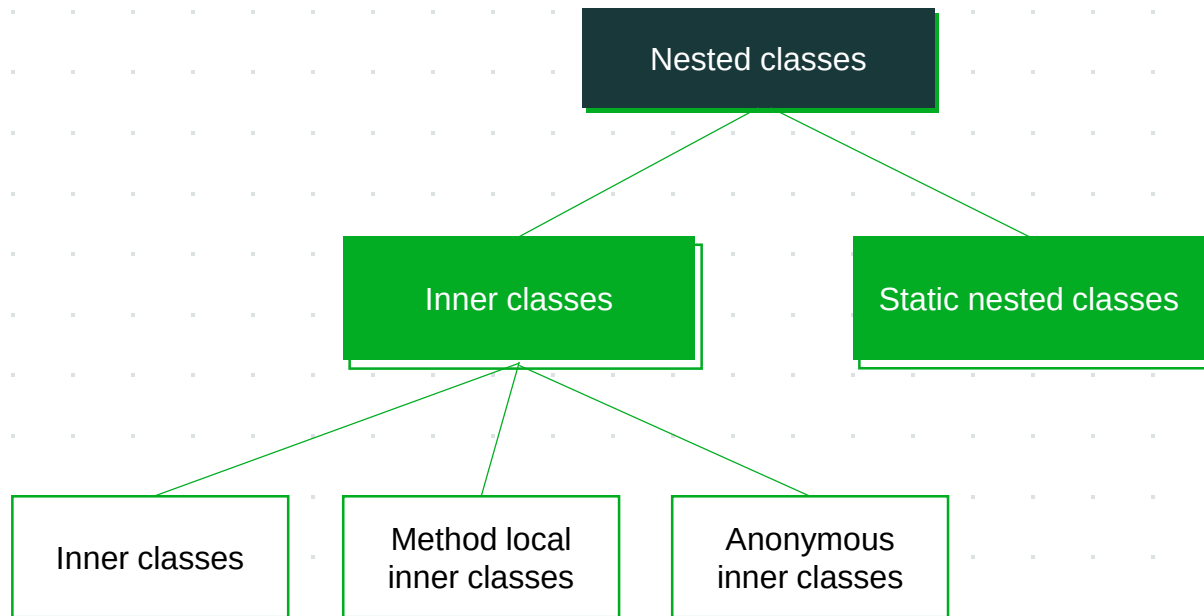
- Вложенные классы
- Абстрактные классы
- Интерфейсы
- Перечисления

nexign

Вложенные классы



Вложенные классы (nested classes)



Вложенный класс (Inner class)

```
class Outer {  
    int field = 42;  
    class Inner {  
        public void show() {  
            //есть доступ к состоянию внешнего класса!  
            System.out.println(field);  
            //печатает 42  
        }  
    }  
    void initInner(){  
        //инициализация вложенного класса внутри  
        new Inner();  
    }  
}  
  
//инициализация вложенного класса извне  
Outer.Inner in = new Outer().new Inner();
```

Inner классы не могут содержать статических методов.



Каждый экземпляр Inner имеет неявную ссылку на Outer

Локальные вложенные классы (method local)

```
class Outer {  
    void outerMethod() {  
        //final (или effectively final)  
        final int x = 98;  
        System.out.println("inside outerMethod");  
        class Inner {  
            void innerMethod() {  
                System.out.println("x = " + x);  
            }  
        }  
        Inner y = new Inner();  
        y.innerMethod();  
    }  
}
```

- Объект локального класса не может создаваться за пределами метода или блока, в котором его объявили.
- Не может использовать локальные переменные внешнего метода до тех пор, пока они не будут объявлены как **final**
- Не может быть помечен как **private**, **protected**, **static**, но может быть помечен как **abstract** и **final**, но не оба одновременно

Анонимные классы (Anonymous)



- анонимные классы не имеют названия
- декларируются внутри методов основного класса и могут быть использованы только внутри этих методов
- должен наследовать существующий класс или реализовывать существующий интерфейс
- не могут содержать определение статических полей, методов и классов (кроме констант), но могут их наследовать

Применение анонимных классов

- Тело класса является очень коротким;
- Нужен только один экземпляр класса;
- Класс используется в месте его создания или сразу после него;
- Имя класса не важно и не облегчает понимание кода.

```
. . .  
button.onClick(new EventListener() {  
    void onClick(Event e) {  
        //здесь у нас доступ ко всем внешним полям  
        //и effectively final-переменным  
    }  
});
```

Статические вложенные классы (static nested)

- Статические вложенные классы могут получить доступ ко всем статическим членам внешнего класса, включая приватные
- Спецификация Java не позволяет объявлять класс верхнего уровня статическим. Только классы внутри других классов могут быть статическими

Статические вложенные классы (static nested)

```
class Outer {  
    private static void outerMethod() {  
        System.out.println("inside outerMethod");  
    }  
    static class Inner {  
        public static void main(String[] args) {  
            System.out.println("inside inner class Method");  
            outerMethod();  
        }  
    }  
}  
.  
.  
.  
Outer.Inner x = new Outer.Inner();
```

Пример реализации – паттерн «singleton»

```
public class Singleton {  
    private Singleton() {}  
  
    private static class SingletonHolder {  
        private static final Singleton INSTANCE = new Singleton();  
    }  
  
    public static Singleton getInstance() {  
        return SingletonHolder.INSTANCE;  
    }  
}
```

Singleton

это порождающий паттерн, который гарантирует существование только одного объекта определённого класса, а также позволяет достигаться до этого объекта из любого места программы

nexign

Абстрактные классы

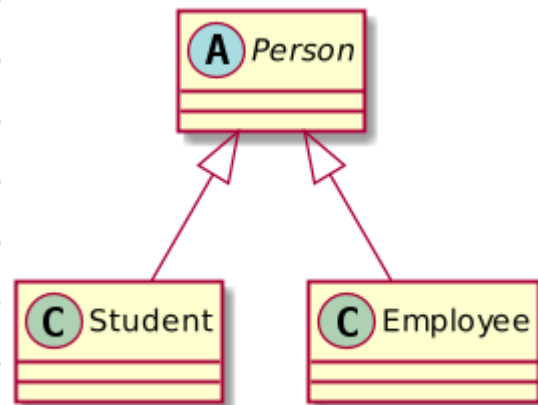


Абстрактные классы

- Класс, в котором хотя бы один из методов не реализован, должен быть помечен как **abstract**
- Нереализованные методы в классе возникают двумя способами:
 - явно объявлены как **abstract**
 - унаследованы из абстрактных классов или интерфейсов и не переопределены.
- Абстрактные классы нельзя инстанцировать через **new**.
 - **new Person("John Doe");** — ошибка компиляции: 'Person is abstract, cannot be instantiated'.



```
public abstract class Person
{
    public Person(String name) {
        this.name = name;
    }
    public String getName() {
        return name;
    }
    public abstract String getDescription();
}
```



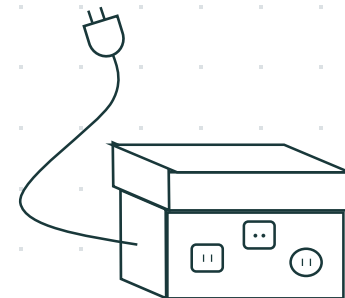
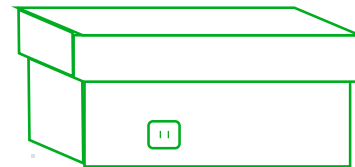
nexign

Интерфейсы



Интерфейсы

- **Интерфейс** – это набор абстрактных методов и статических констант
- Синтаксически определение интерфейса похоже на определение **полностью** абстрактного класса, содержащего заголовки методов и **static final** поля
- Для реализации интерфейса класс должен создать полный набор методов, определенных интерфейсом. Однако каждый класс волен сам определять детали своей реализации.
- Создавать объект типа «интерфейс» нельзя, но можно использовать в наследовании
- Как только интерфейс определен, то реализовать его может любое число классов. И наоборот: один класс может реализовать любое число интерфейсов.



Интерфейсы

```
//его нельзя инстанцировать!  
public interface Prism  
{  
    //это --- final-переменная!  
    double PI = 3.14;  
  
    //это --- публичные абстрактные методы!  
    double getArea();  
    double getHeight();  
  
    //этот метод может вызывать другие и читать  
    final-поля  
    default double getVolume() {  
        return getArea() * getHeight();  
    }  
}
```

Реализация интерфейса

Если какой-то из методов интерфейса не будет переопределён, класс нужно пометить как `abstract`.

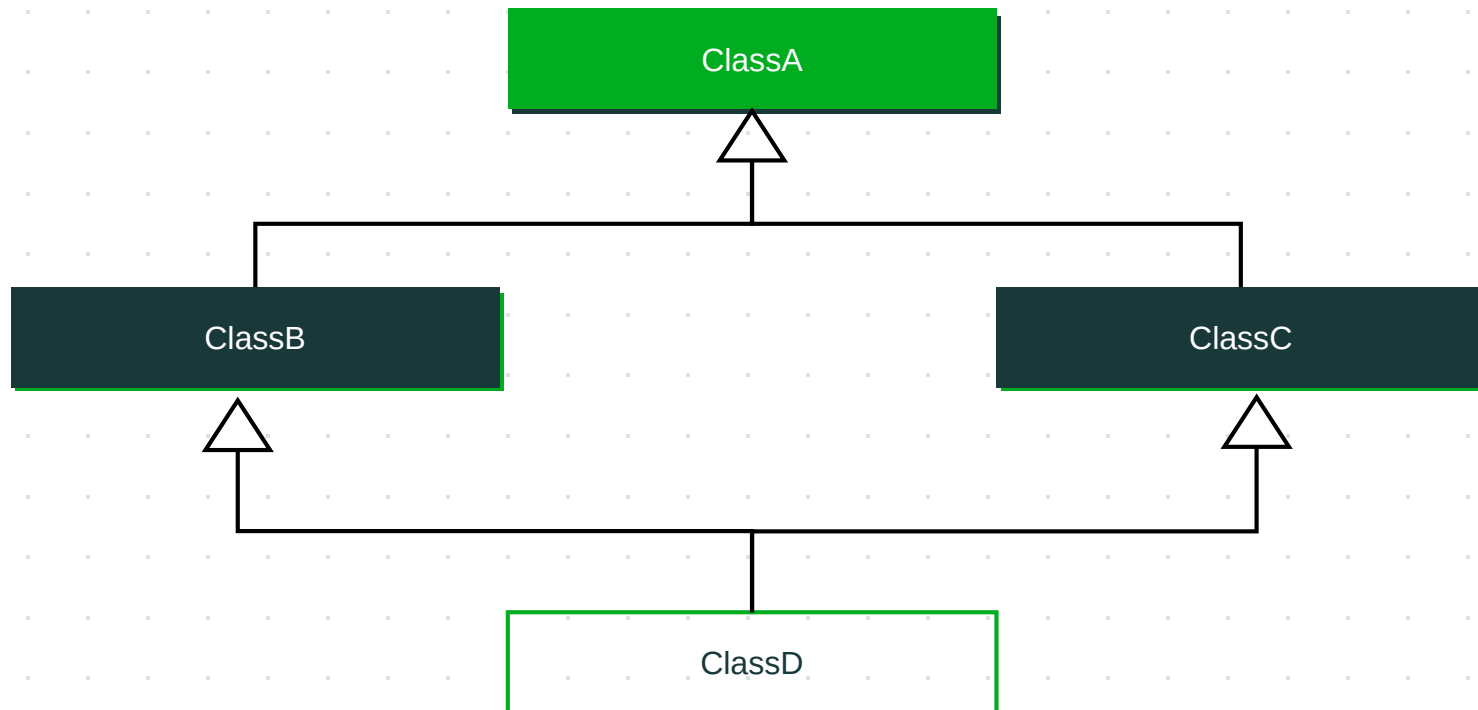
```
public class Parallelepiped implements Prism {  
    private double a;  
    private double b;  
    private double h;  
    @Override  
    public double getArea() {  
        return a * b;  
    }  
  
    @Override  
    public double getHeight() {  
        return h;  
    }  
}
```

Абстрактный класс

- Описывает концептуальный объект
- Abstract
- Extends
- Наследуется только один класс
- Есть конструктор
- Переменные с любыми модификаторами
- Любые модификаторы методов
- все методы кроме абстрактных должны иметь реализацию
- Может наследовать обычные и абстрактные классы, а так же реализовывать интерфейсы

Интерфейс

- Описывает поведение
- Interface
- Implements
- Реализуется множество интерфейсов
- -
- Все переменные неявно public static final
- Для методов только public и private(Java 9+ *)
- Default –методы должны иметь реализацию в интерфейсе и могут быть переопределены в имплементации
- Может наследовать множество других интерфейсов



Множественное наследование

```
public abstract class ClassA {  
    public abstract void doSomething();  
}
```



```
public class ClassB extends ClassA{  
    @Override  
    public void doSomething(){  
        //Какая-то реализация класса B  
    }  
    //собственный метод класса ClassB  
    public void methodB() {  
    }  
}
```

```
public class ClassC extends ClassA{  
    @Override  
    public void doSomething(){  
        //Какая-то реализация класса C  
    }  
    //собственный метод класса ClassC  
    public void methodC() {  
    }  
}
```

```
public class ClassD extends ClassB, ClassC{  
    public void test(){  
        //вызов метода родительского класса  
        doSomething();  
    }  
}
```

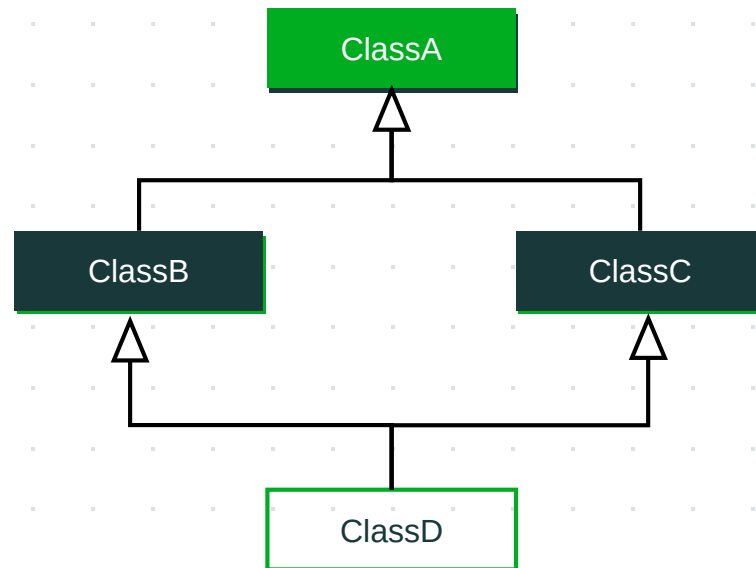
Ромбовидное наследование

#1

```
interface ClassA { . . . }  
interface ClassB extends ClassA { . . . }  
class ClassC implements ClassA { . . . }  
class ClassD extends ClassC implements ClassB  
{ . . . }
```

#2

```
interface ClassA { . . . }  
interface ClassB extends ClassA { . . . }  
interface ClassC extends ClassA { . . . }  
class ClassD implements ClassB, ClassC { . . . }
```



nexign

Перечисления





- Перечислимый тип – это список именованных констант
- Метод **values()** возвращает список констант перечислимого типа.
- Метод **valueOf()** возвращает константу перечислимого типа по значению строки-параметра.
- **ENUM_VALUE.name()** – возвращает имя (**=toString()**)
- **ENUM_VALUE.ordinal()** – возвращает порядковый номер
- Можно определять конструкторы, добавлять поля и методы, реализовывать интерфейсы.

Q&A

